Graph-Guided LLM Fault Localisation: An Empirical Study on SQLite

PAKHI DEBNATH-240101065, ADITI MALAV-240101007, IIT Guwahati, India

Call-graph-guided retrieval is a natural fit for LLM-based fault localisation: a structured graph constrains the search space and can either be navigated via tool calls or serialised as a text slice in the prompt. We evaluate whether two call-graph representations of SQLite 3.30.1—a compile-time graph extracted with Clang 18.1.3 and a syntactic graph extracted with tree-sitter—improve LLM fault-localisation accuracy for six benchmark questions on two large language models (Qwen3-Coder-30B and Nemotron-3-Ultra-550B). We compare a tool-using agent that navigates the graph via function calling, a text-slice condition that injects a BFS subgraph into the user message, and a no-context baseline, together with four tool-ablation variants. Graph context rarely improves exact accuracy under the 15-turn baseline: of the 43 wrong answers across all eight graph conditions, 15 (35 %) stem from function-pointer callback gaps that make the target unreachable along static call edges, and 25 (58 %) name a different graph node. Text slices match the 15-turn tool condition in exact accuracy (and are within one question in top-3 coverage) at about ten to twelve times lower token cost for Qwen3-Coder-30B and sixteen to twenty-three times lower for Nemotron-3-Ultra-550B. Neither graph representation (compile-time vs. syntactic) demonstrated superior accuracy. In sensitivity probes (designed after the baseline results were known), a 30-turn budget with a revised prompt allowed agents to sometimes locate callback-separated targets, at steep token expense.

CCS Concepts: • **Software and its engineering → Software testing and debugging**; *Automated static analysis*.

Additional Key Words and Phrases: fault localisation, call graph, large language models, LLM agents, static analysis, SQLite, empirical study, function calling

1 Introduction

Identifying which function is responsible for a reported bug—fault localisation—is one of the most time-consuming steps in software maintenance [Wong et al. 2016]. Traditional automated approaches such as spectrum-based FL (SBFL) rank functions by test-coverage patterns [Jones and Harrold 2005], requiring both a test suite and the ability to execute the program. Large language models (LLMs) offer a complementary capability: they can reason over natural-language bug descriptions and source code simultaneously, and can be equipped with *tools* that retrieve task-relevant information on-the-fly [Yao et al. 2023]. However, LLMs cannot read an entire production codebase within their context window, so retrieval quality is a key bottleneck.

A code call-graph provides a structured, navigable index of which functions call which others. Given a call-graph, a tool-using agent can iteratively narrow the search— following callers upward, callees downward, and taint paths through the data-flow—without loading the full source. Alternatively, a breadth-first slice of the same graph can be serialised as plain text and injected into the prompt, trading repeated tool-call overhead for a single, larger context window. Both strategies have been proposed in recent work [Li et al. 2026; Yildiz et al. 2025], but to our knowledge, among the work we reviewed, a direct controlled comparison on a real, well-known codebase under identical conditions is lacking.

We build two call-graphs for the SQLite 3.30.1 source: a compile-time graph extracted with Clang 18.1.3 (417 functions, 2,438 call edges, 84 taint edges (41 distinct pairs)) and a syntactic graph extracted with tree-sitter/graphify. We then run a controlled experiment across five conditions

Author's Contact Information: Pakhi Debnath-240101065, Aditi Malav-240101007, IIT Guwahati, India.

FSE (CS-5187 Term Project), IIT Guwahati
2026.

(no-context, tool-agent on each graph, text-slice on each graph) and four tool-ablation variants, using two instruction-tuned LLMs (Qwen3-Coder-30B and Nemotron-3-Ultra-550B) on six fault-localisation benchmark questions.

Research questions.

RQ1 Does call-graph context (tool or text) improve fault-localisation accuracy over a no-context baseline?

RQ2 Does compiler-grade compile-time (Clang) extraction outperform syntactic (tree-sitter) extraction?

RQ3 Does tool-based graph navigation yield higher accuracy than delivering the same graph as a text slice?

RQ4 Which graph-navigation tools are necessary for accuracy?

RQ5 How often do graph-navigating LLMs cite call edges not supported by the underlying graph?

Main findings.

(1) Graph context provides only marginal accuracy gains in the 15-turn baseline: neither model exceeds one correct answer out of five (excluding Q2) or six (Nemotron). Furthermore, 15 of 48 answer slots (31 %) represent callback gaps where targets are unreachable along static call edges.
(2) Neither graph representation demonstrated superiority: CT and SYN achieved identical exact accuracy in the TEXT conditions (excluding Q2) and differed by one question in the TOOL conditions (Qwen3-Coder-30B: SYN 1, CT 0; Nemotron-3-Ultra-550B: 1 each).
(3) Text slices match or exceed 15-turn tool conditions in exact accuracy at about ten to twelve times lower token cost for Qwen3-Coder-30B and sixteen to twenty-three times lower for Nemotron-3-Ultra-550B. In top-3 accuracy, tool conditions are at least as good as text slices in all four comparisons, and strictly better in three.
(4) Source-only retrieval (no edge-traversal tools) consumes 40–62× more tokens than text slices with no accuracy gain; caller/callee tools were associated with a 2.4-fold lower rate of edges not supported by the graph (12 % vs. 28 %).

Contributions.

(1) Two open call-graph databases for SQLite 3.30.1 (compile-time and syntactic), together with graph query tool suites (7 tools for CT, 5 for SYN), a text-slice generator, and a resumable multi-condition evaluation harness.
(2) An empirical study comparing tool-based and text-slice context delivery across two LLMs and nine experimental conditions, with a detailed error classification and path-correctness analysis.
(3) An annotated dataset of 48 answer slots (including 43 failure slots) and 133 edge-verification records identifying when graph-conditioned LLMs cite edges absent from the graph.

2 Background

Fault localisation. Automated fault localisation (FL) aims to rank program elements—statements, methods, or functions—by their likelihood of containing a defect. Spectrum-based FL (SBFL) methods such as Tarantula [Jones and Harrold 2005] compute suspiciousness scores from pass/fail test-coverage data. A broad survey of FL techniques is given by Wong et al. [2016]. Our work differs from SBFL: we ask an LLM to identify the faulty function from a natural-language bug description and optional call-graph context, without access to test suites or execution traces.

LLM agents with tool use. The ReAct framework [Yao et al. 2023] interleaves reasoning steps with calls to external tools and observations of their results. OpenAI-compatible APIs expose this pattern via a *function-calling* interface: the model selects from a schema-defined tool set and the host dispatches the call. We use this interface to let the LLM navigate a call-graph database without loading the full graph into the prompt.

Call graphs. A call graph records direct-call edges between functions in a program. A *compile-time* (CT) call graph is extracted by parsing source with a compiler front-end, capturing precise call sites with source-level provenance. A *syntactic* (SYN) call graph is extracted by pattern-matching on the parse tree without full compilation, trading precision for ease of construction. Neither representation models function-pointer calls, creating structural gaps when control flow crosses a pointer boundary.

3 Problem Statement

We study the following *function-level fault localisation* task: given (i) an entry function from which the bug manifests, (ii) a natural-language description of the observed symptom, and (iii) optional access to a call-graph of the target codebase, identify the single function most likely to be faulty.

Target program. We use SQLite 3.30.1 (commit `25e9591`, tag `version-3.30.1`), a widely studied embedded SQL engine [SQLite Development Team 2024]. We extract call graphs from eleven core C source files spanning tokenization, parsing, query planning, expression resolution, and virtual machine execution (348,852 tokens, about 31,000 lines of C). SQLite's source is public and its dispatch relies on function pointers, which exposes the structural-gap problem studied here; its bugs are also public, which we discuss as a threat to validity (Section 7).

Benchmark. We evaluate on six fault-localisation questions drawn from real SQLite bug fixes committed after version 3.30.1. Each bug was reproduced on the 3.30.1 snapshot with call chains manually verified. Following decision-log rule D14, the ground truth is strictly defined as the *fault site*—the function modified in the upstream fix patch—rather than the downstream crash or assertion site. For Q3 and Q4 the bug was reproduced on the vulnerable build, but the fix-build differential was not run; ground truth follows D14 and was agreed by both authors (for Q4 the upstream fix also modifies `sqlite3ErrorMsg`, which is outside the analysed files). Prompts use symptom-only wording and name only the entry function, never the answer function (for Q2 the two coincide, see §5.2). The target functions span SQL query processing (`sqlite3Select`, `flattenSubquery`, `withExpand`), string formatting (`sqlite3_str_vappendf`), expression ordering (`resolveOrderByTermToExprList`), and aggregate accumulation (`resetAccumulator`).

Structural challenge and callback gaps. SQLite relies extensively on function pointers stored in walker structs and callback tables. Neither the compile-time nor the syntactic graph models dynamic function-pointer dispatches. Consequently, two targets—`resolveOrderByTermToExprList` (Q1) and `withExpand` (Q4)—are present as nodes in both graph databases but are unreachable from the entry function along static call edges, because control reaches them through function pointers. A third question (Q5, `sqlite3_str_vappendf`) also executes through a function pointer at runtime; in the static call graph, it is topologically reachable only via an unrelated error-logging route rather than the active data path. These gaps bound what pure edge traversal can reach; however, an agent equipped with symbol search and source reading can in principle locate them if granted sufficient search budget.

4 Approach

4.1 Graph Construction

Compile-time (CT) graph. We parse eleven SQLite C files with Clang 18.1.3 via libclang, recording every direct function call with its source location, argument identifiers, and return-value assignments. The resulting graph CT-v1 has 417 defined functions, 392 external (boundary) functions, 94 struct/union types, 954 fields, and 2,438 call-site edges (1,593 distinct caller–callee pairs, including calls into external functions) stored in a SQLite database using a shared schema (`nodes` and `edges` tables). Forty indirect (function-pointer) calls are recorded in node metadata but not as edges. A lightweight taint pass propagates three source patterns (`sqlite3_prepare*`, `sqlite3_bind_text*`, `sqlite3_bind_blob*`) from 11 source functions to the sink functions in three classes (copy, alloc, free); of the 16 sinks defined, 6 are reached. Depth-limited search ($K = 6$) adds 84 `taint_reaches` edge records (41 distinct source–sink pairs) to produce CT-v2. Struct and union fields are stored separately as `has_field_of_type` edges for type-hierarchy queries. The taint pass is a purely syntactic over-approximation: it does not model aliasing, pointer arithmetic, or global state. Taint edges were not hand-validated on the final graph and no result relies on them.

Syntactic (SYN) graph. We run the `graphify` tool (tree-sitter back-end) on the same eleven files and convert the resulting JSON to the shared schema via an adapter script. The resulting graph contains 440 functions and 772 call edges (of which 310 were inferred by heuristic). Unlike Clang which models each distinct call site, `graphify` records unique caller–callee pairs. Furthermore, `graphify` failed to parse macro-heavy header definitions, so it did not extract struct fields or taint information; thus the SYN graph does not support type-hierarchy or taint queries. End-line values missing from the tree-sitter output are recovered via a brace-matching or next-definition heuristic.

4.2 Tool Set

The two graphs expose different tool subsets (Table 1). CT supports all seven tools. SYN supports five, because it has no type or taint edges. Results are sorted and capped at 50 items; source output is capped at 8,000 characters (roughly 100–200 lines) per call, with paging for longer functions. All tools return `{"result": ..., "truncated": bool}` or `{"error": CODE, ...}` and are exposed to the LLM as OpenAI function-calling schemas.

Table 1. Graph-navigation tools and their availability per graph.

Tool	Description	CT	SYN
`get_callers(fn)`	Direct callers of a function	✓	✓
`get_callees(fn)`	Direct callees of a function	✓	✓
`find_call_path(src, dst)`	BFS shortest call chain	✓	✓
`get_function_source(fn)`	Paginated source lines	✓	✓
`search_functions(substr)`	Fuzzy name search	✓	✓
`get_type_hierarchy(type)`	Struct/union fields and types	✓	—
`get_taint_paths(param)`	Sinks reachable from tainted param	✓	—

4.3 Agent Loop (TOOL Conditions)

The agent runs up to 15 turns via the OpenAI-compatible function-calling interface at temperature 0. On each turn the model either calls a tool (observation appended to history) or emits a final JSON answer conforming to the ANSWER schema (required fields: `answer_function`, `top3`,

`evidence_path`, `taint_path`, `cited_edges`, `explanation`). If no final answer is produced within 15 turns, a force-prompt is appended and one additional call is made without tools (`stopped_by = forced_final`). All 48 tool-condition runs—36 from Qwen3-Coder-30B (12 non-ablation, 24 ablation) and 12 from Nemotron-3-Ultra-550B—hit the 15-turn cap; every reported tool-condition answer is therefore a forced final answer.

4.4 Text Slice (TEXT Conditions)

A breadth-first search from the entry function over `calls` edges serialises the call neighbourhood as text, one block per function, up to $B = 6{,}000$ tokens (tiktoken `cl100k_base`). Blocks are added whole and the slice stops before the first one that would exceed the ceiling. Five questions share `sqlite3Select` as entry, so they share one slice (2,849 tokens on SYN, 4,405 on CT, both complete). Q5 uses `sqlite3_step` (2,963 tokens on SYN, 5,982 tokens on CT, cut by the ceiling). The slice is prepended to the user message, before the question text. The model answers in one turn without tool access.

4.5 Evaluation Design

Table 2 summarises the nine conditions. NOCTX provides neither code nor tools and is repeated three times per question at temperature 0.7 to estimate baseline variance. All other conditions use temperature 0 and run once per question. Ablation conditions (last four rows) were run on Qwen3-Coder-30B only: the NVIDIA API returned malformed or empty completions for Nemotron-3-Ultra-550B on every tool-subset call; this problem did not manifest in the full tool-set conditions.

Table 2. Experimental conditions. "5 tools" denotes the SYN-compatible subset; "7 tools" includes type-hierarchy and taint paths (CT only); "src+search" denotes `get_function_source` and `search_functions`.

Condition	Graph	Delivery	Tools
NOCTX	—	None	None
SYN-TOOL	SYN	Tool calls	5 tools
CT-TOOL	CT-v2	Tool calls	7 tools
SYN-TEXT	SYN	Text slice	None
CT-TEXT	CT-v2	Text slice	None
CT-CT_CALLS	CT-v2	Tool calls	4 (callers, callees, source, search)
CT-CT_TYPES	CT-v2	Tool calls	5 (+ type hierarchy)
CT-SRC-ONLY	CT-v2	Tool calls	2 (src+search)
SYN-SRC-ONLY	SYN	Tool calls	2 (src+search)

5 Evaluation

5.1 Models

We evaluate two instruction-tuned LLMs: Qwen3-Coder-30B (Qwen3-Coder-30B-A3B-Instruct, served locally via LM Studio) and Nemotron-3-Ultra-550B (nvidia/Nemotron-3-Ultra-550B-A55B, accessed via the NVIDIA Inference API with `enable_thinking: false`). Both support OpenAI-compatible function calling.

5.2 RQ1: Does Graph Context Improve Accuracy?

Table 3 reports exact and top-3 accuracy per condition. *Exact* counts runs where `answer_function` matches the ground truth; *top-3* counts runs where the ground truth appears in `top3` or equals

`answer_function`. For Question 2, the ground truth is `sqlite3Select`, which is also the entry function named in the question prompt; answering the entry function is therefore correct by construction. Because Q2's success reflects this benchmark artefact (and was correct in 3 of 3 NOCTX runs on Qwen3-Coder-30B), we report accuracy both with Q2 excluded (n=5, primary analysis) and with Q2 included (n=6) across both models for transparency.

Table 3. Accuracy per condition. Primary analysis excludes Q2 (n = 5); full benchmark includes Q2 (n = 6). Dashes denote ablation subsets not evaluated on Nemotron due to API failures. NOCTX exact and top-3 counts reflect questions with ≥1 hit across 3 repeats.

	Qwen (n=5)		Qwen (n=6)		Nemotron (n=5)		Nemotron (n=6)	
Condition	**Ex.**	**T3**	**Ex.**	**T3**	**Ex.**	**T3**	**Ex.**	**T3**
NOCTX	0	0	1	1	1	1	1	2
SYN-TOOL	1	2	1	3	1	4	1	4
CT-TOOL	0	1	0	2	1	2	1	3
SYN-TEXT	1	1	1	1	1	2	1	3
CT-TEXT	1	1	2	2	1	1	1	2
CT-CT_CALLS	0	1	0	2	—	—	—	—
CT-CT_TYPES	0	1	0	2	—	—	—	—
CT-SRC-ONLY	0	1	1	2	—	—	—	—
SYN-SRC-ONLY	0	1	0	2	—	—	—	—

Overall exact accuracy is low across all baseline conditions: no condition exceeds one exact answer out of five in the primary analysis (or two out of six on Qwen CT-TEXT when Q2 is included). In the no-context baseline, excluding Q2 yields 0 of 15 exact runs on Qwen3-Coder-30B and 1 of 15 on Nemotron-3-Ultra-550B (Q1); including Q2 yields 3 of 18 exact runs on Qwen3-Coder-30B (all Q2) and 1 of 18 on Nemotron-3-Ultra-550B. Providing graph context does not reliably improve exact accuracy in the 15-turn baseline.

5.3 RQ2: Compile-Time vs. Syntactic Graph Extraction

Under the 15-turn baseline the two graphs differ by at most one question. Excluding Q2, SYN-TEXT and CT-TEXT each score 1 of 5; on Qwen3-Coder-30B, SYN-TOOL scores 1 and CT-TOOL 0; on Nemotron-3-Ultra-550B both score 1. We find no detectable difference; with six questions we cannot claim equivalence.

5.4 RQ3: Tool vs. Text Slice

In exact accuracy, text slices match or exceed tool conditions in the 15-turn baseline: for Qwen3-Coder-30B (n=5), SYN-TEXT and CT-TEXT each score one exact answer (Q5), matching SYN-TOOL (Q1) and exceeding CT-TOOL (zero exact). For Nemotron-3-Ultra-550B, all four graph conditions achieve one exact answer (Q5). In top-3 accuracy, tool conditions are at least as good as text slices in all four comparisons, and strictly better in three (e.g., Nemotron SYN-TOOL achieves 4 of 6 top-3 hits vs. 3 of 6 for SYN-TEXT, and Qwen SYN-TOOL achieves 2 of 5 vs. 1 of 5 for SYN-TEXT), though these differences represent only single-question margins.

Note that TOOL and TEXT conditions differ in more than delivery mode: in the TOOL conditions the agent can call `get_function_source` to read raw source, whereas in the TEXT conditions the slice is the only source representation and raw source is not separately available. This confound is discussed further in Section 7.

Token cost and latency (Table 4). Across both models, text slices consume between 3,400 and 5,500 mean prompt tokens (3,401 for Qwen SYN-TEXT; 5,480 for Nemotron CT-TEXT). In contrast, full tool conditions cost approximately 10–12× more for Qwen3-Coder-30B (39,474–54,442 tokens) and 16–23× more for Nemotron-3-Ultra-550B (81,796–86,373 tokens). All 48 baseline tool-condition runs hit the 15-turn cap and ended with a forced final answer, fully consuming the turn budget.

Table 4. Measured prompt tokens, completion tokens, and latency across all conditions for both models. Tool-condition prompt tokens are cumulative across turns (not a single context window). Dashes denote ablation conditions not evaluated on Nemotron.

	Qwen3-Coder-30B			Nemotron-3-Ultra-550B		
Condition	**Prompt**	**Compl.**	**Time (s)**	**Prompt**	**Compl.**	**Time (s)**
NOCTX	324	232	2	342	263	11
SYN-TOOL	39,474	710	16	81,796	908	54
CT-TOOL	54,442	687	17	86,373	796	48
SYN-TEXT	3,401	276	4	3,534	708	19
CT-TEXT	5,208	361	6	5,480	282	7
CT-CT_CALLS	67,778	708	20	—	—	—
CT-CT_TYPES	60,796	702	17	—	—	—
CT-SRC-ONLY	208,833	759	49	—	—	—
SYN-SRC-ONLY	210,621	804	55	—	—	—
SYN-TOOL (30-TURN)	286,523	1,446	58	—	—	—
CT-TOOL (30-TURN)	270,512	1,213	51	—	—	—

5.5 Error Analysis

To understand failure modes, every Qwen3-Coder-30B answer across all eight graph conditions (48 total answer slots across 6 questions; NOCTX excluded) was classified. Five slots contain correct answers (1 on SYN-TOOL, 1 on SYN-TEXT, 2 on CT-TEXT, and 1 on CT-SRC-ONLY); the remaining 43 wrong answers fall into one of four buckets, evaluated in priority order:

Callback gap (15 of 43 wrong answers, 35 %; 15 of 48 total slots). The ground-truth function is present in the graph database as a node, but is unreachable from the entry along static `calls` edges because control flow crosses a function-pointer boundary. This accounts for 7 of 8 wrong conditions on Q1 and all 8 wrong conditions on Q4. The static graph slice also lacks the ground-truth function for both questions.

Wrong other (25 of 43 wrong answers, 58 %; 25 of 48 total slots). The ground-truth function is present as a node in the graph database, but the model named a different function. For Q3 (`flattenSubquery`), the graph contains the ground truth yet the model named `codeDistinct` across six of eight conditions. For Q5 (`sqlite3_str_vappendf`) on Qwen3-Coder-30B, every tool condition stopped at `printfTempBuf` (a callee of `sqlite3_str_vappendf`); the text slice surfaced the target at shallow BFS depth.

Default to entry (3 of 43 wrong answers, 7 %; 3 of 48 total slots). The model named the entry function itself (`sqlite3Select` or `sqlite3_step`). Answers naming `sqlite3Select` on Q3 and Q6 fall into this category because `sqlite3Select` is the entry function for those queries.

Not in graph (0 of 48 slots). Every ground-truth function is present as a node in both graph databases.

5.6 RQ4: Tool Ablation

No ablation condition scored above zero exact answers on Qwen3-Coder-30B's five unflagged questions (Table 3), and every ablation run hit the 15-turn cap. Adding callers, callees, type hierarchy, and taint paths to the source-reading baseline did not improve exact accuracy on these five questions. However, the source-only control incurred severe token inflation: consuming an average of 208,833 tokens on CT and 210,621 tokens on SYN, source-only cost roughly 4× more than full tool navigation (54,442 tokens on CT) and 40–62× more than text slices (5,208 on CT-TEXT, 3,401 on SYN-TEXT). Without callers or callees to narrow the candidate space, the model largely paged through raw source lines until hitting the iteration cap.

5.7 RQ5: Edge-Citation Validity

Every cited edge (`src`, `dst`, `kind`) and every consecutive evidence-path pair from all 36 Qwen3-Coder-30B tool-condition runs were verified against the condition's own graph database (CT conditions against `graph_ct_v2.db`; SYN conditions against `graph_syn.db`). An edge is recorded as *not supported by the graph* when neither forward nor reverse direction appears in the graph; because neither graph models function-pointer calls, an absent edge may reflect a genuine structural gap rather than model fabrication, so this rate represents an upper bound on hallucination.

Of 133 cited edges, 106 (80 %) are present exactly as stated and 4 (3 %) have direction reversed. The rate of edges not supported by the graph was 2.4× lower in conditions with caller/callee tools (12 %, 10 of 86 edges) than in source-only conditions (28 %, 13 of 47 edges). Evidence-path pairs show a similar trend (14 % vs. 35 %). This descriptive association indicates that edge-traversal tools ground the agent's cited reasoning paths to verifiable edges, whereas source-only agents frequently cite edges not supported by the underlying call graph. Because these tools return graph edges, citing in-graph edges is expected by construction; the gap mainly shows that source-only agents infer edges the graph does not hold. Notably, the single Qwen3-Coder-30B run that correctly answered Q1 under the 15-turn baseline (SYN-TOOL) still cited an edge not supported by the graph, spanning the function-pointer gap; an exact answer does not ensure a fully verified reasoning chain.

5.8 Turn Horizon and Guidance Sensitivity Analysis

In the baseline protocol, all 48 tool runs hit the 15-turn cap and required a forced final answer. To understand the underlying causes, we conducted an exploratory sensitivity analysis examining turn awareness, stopping cues, and search horizon on Qwen3-Coder-30B. Per decision-log entry D17 (written after the baseline results were known), all modifications are evaluated on a separate run track and reported side by side. Because the prompt, turn cap, and temperature changed together, the performance changes below cannot be attributed to any single factor in isolation.

Turn awareness and early stopping (15 turns). We updated the tool prompt to provide a per-turn countdown (*"Tool calls left: N"*) and a convergence nudge at two-thirds budget (*"Start converging. If you have a candidate, answer now"*). Under this 15-turn configuration, 4 of 12 runs concluded with `stop=final` on their own without invoking the forced final prompt. These 12 runs scored 0 of 12 exact.

Extended search horizon with 3 repetitions (30 turns, $T = 0.7$). To evaluate whether performance gains were stable or stochastic, we ran 3 sampled repetitions per question at temperature 0.7 with a 30-turn cap and the refined guidance prompt (36 runs total). The results show:

- **Autonomous termination**: Rather than hitting the turn cap indiscriminately, **26 of 36 runs (72 %)** terminated autonomously with `stop=final`, with several runs concluding between turns 22 and 26.
- **Callback-gap targets reached**: On CT-TOOL, `resolveOrderByTermToExprList` (Q1) was solved in **2 of 3 repetitions** (67 % consistency). Notably, `withExpand` (Q4)—which was never solved by any condition or model in the 15-turn baseline—was identified on **both graphs** (1 of 3 runs on SYN-TOOL, and 1 of 3 runs on CT-TOOL).
- **Overall accuracy**: Excluding Q2, per-run exact accuracy was 20 % on both graphs (3 of 15 each). The 15-turn baseline was 1 of 5 on SYN-TOOL and 0 of 5 on CT-TOOL, so the overall rate barely moved. What changed was which questions were solved, notably Q4, which no baseline condition solved. Across the full 6 questions, top-3 coverage reached 100 % on Q2 (both graphs) and Q5 (SYN-TOOL), and 67 % on Q1 and Q5 (CT-TOOL).

Goal-wording control experiment. To verify whether performance gains in the 30-turn probe were driven by prompt phrasing rather than active tool navigation, we evaluated a control condition applying the updated goal wording to NOCTX, SYN-TEXT, and CT-TEXT on Qwen3-Coder-30B (30 runs). The goal wording alone failed to solve Q1 or Q4 in any condition: both text slices scored 0 of 1 on Q1 and Q4, and NOCTX defaulted to answering `sqlite3Select` on 5 of 6 questions. This suggests that discovering `resolveOrderByTermToExprList` (Q1) and `withExpand` (Q4) in the extended tool runs was aided by interactive tool traversal and source reading, rather than prompt phrasing alone.

Methodological takeaways. Tuning guidance prompts on the evaluation benchmark carries an inherent risk of optimistic bias, and the additional navigation depth incurs substantial token cost (averaging 270,512 prompt tokens on CT and 286,523 on SYN). Furthermore, on a six-question benchmark, the 3 of 15 per-run accuracy on both graphs represents a modest absolute count that warrants cautious interpretation. We did not run a 30-turn source-only condition, so we cannot attribute the gains to the graph tools rather than to symbol search and source reading. Nonetheless, the sensitivity experiments suggest that: (i) a longer turn budget together with the revised prompt allowed some callback-separated targets to be reached (the effect of the cap alone was not isolated), (ii) turn countdown visibility was associated with earlier stopping, and (iii) static function-pointer gaps do not represent an insurmountable ceiling when agents can inspect source and search symbol definitions.

6 Discussion

Graph completeness and tool interaction. The 31 % of baseline slots attributable to callback gaps illustrates the penalty imposed when static call graphs omit function-pointer dispatches. Under a standard 15-turn traversal, targets separated by callback boundaries remained unreached. However, our sensitivity analysis revealed that graph tools do not operate in a vacuum: given an expanded 30-turn budget, CT-TOOL combined graph exploration with symbol searches and source inspection to locate both Q1 and Q4. Because we did not evaluate a 30-turn source-only baseline, we cannot attribute these gains specifically to the graph tools rather than to symbol search and raw source reading. Thus, static graph gaps bound what pure edge traversal can reach, though hybrid agents equipped with source inspection can partially bridge them at substantial token expense.

Compile-time vs. syntactic call graphs. A central empirical question is whether compiler-grade extraction (Clang) outperforms lightweight syntactic extraction (tree-sitter). Our results show no detectable difference in fault-localisation accuracy: CT and SYN achieved identical exact accuracy in the TEXT conditions (excluding Q2) and differed by only one question in the TOOL conditions

(Qwen3-Coder-30B: SYN 1, CT 0; Nemotron-3-Ultra-550B: 1 each). Clang recorded 2,438 call-site edges (1,593 distinct caller–callee pairs, including calls into external library functions) compared to 772 unique pairs in `graphify`; the CT count includes boundary functions and the two counts are not strictly comparable, but the additional compiler precision did not translate into higher localisation accuracy on this benchmark.

Text slices vs. interactive tool navigation. Text slices match or exceed tool conditions in exact accuracy at a fraction of the token cost (3,400–5,500 tokens vs. 39,474–86,373 for tool conditions; 10–12× for Qwen, 16–23× for Nemotron). In top-3 coverage, tool conditions are at least as good as text slices in all four comparisons, and strictly better in three, though the margin is only a single question. For Q5 on Qwen3-Coder-30B, the BFS slice happens to include `sqlite3_str_vappendf` at shallow depth, while the tool agent stops at a callee one hop deeper (`printfTempBuf`). This suggests that for shallow-target questions where the answer lies within a small BFS radius of the entry, a text slice is a cost-effective alternative.

Caller/callee tools and edge citation validity. The not-in-graph rate was 2.4× lower in conditions with caller/callee tools (12 %) than in source-only conditions (28 %). We report this as a descriptive association on one model (133 edges across 36 runs); no statistical test was applied. Some absent edges reflect function-pointer structural gaps rather than model fabrication, so the rate is an upper bound on actual hallucination. Even so, conditions with edge-traversal tools appear to constrain the model to citing edges that actually exist in the graph, even when those tools do not change the final answer.

Majority consensus and candidate comparison. Taking the 2-of-3 majority gives the correct answer for Q5 on SYN-TOOL and Q1 on CT-TOOL (1 of 5 per graph), no better than the single-run baseline. Three samples are too few to say whether voting helps. Furthermore, qualitative inspection of failed trajectories suggests that prompting the agent to explicitly compare two or three candidate functions before committing—rather than selecting the first symptom-matching name such as `codeDistinct`—might reduce false positives.

7 Threats to Validity

Internal validity. Q2 benchmark artefact and contamination control. For Q2, the ground-truth function (`sqlite3Select`) is identical to the entry function specified in the problem prompt; answering the entry function is therefore correct by construction. In the baseline NOCTX condition, Qwen3-Coder-30B answered `sqlite3Select` in all 3 repeats, and in the goal-wording control run it defaulted to `sqlite3Select` across 5 of 6 questions. To prevent this benchmark artefact from skewing conclusions, we exclude Q2 from the primary comparisons across both models (n=5) and report the full benchmark (n=6) separately. *Training-data contamination.* The six bugs are public and the training cutoffs of both models are unknown, so some answers may reflect memorisation. The no-context runs are a partial control only: excluding Q2, Qwen got 0 of 15 exact and Nemotron 1 of 15, but Nemotron placed the ground truth in its top three for Q1 and Q2 in all three repeats, which may reflect prior knowledge or cues in the question text. *Forced final answers and turn horizon.* All 48 baseline tool-condition runs (36 from Qwen3-Coder-30B, comprising 12 full-tool and 24 ablation runs, and 12 from Nemotron-3-Ultra-550B) hit the 15-turn cap; every baseline tool answer is therefore a forced final answer rather than a model-initiated termination. Our sensitivity probes (Section 5.8) are consistent with turn truncation being a limiting factor, though the prompt revision changed at the same time and the cap alone was not isolated: extending the budget enabled both tool conditions to solve additional targets (notably Q4, previously unsolved) and was associated with earlier autonomous terminations. *Tool delivery*

and source granularity confounds. The comparison between TOOL and TEXT conditions carries a delivery confound: tool-condition agents can read raw function bodies via `get_function_source`, whereas text-condition models see only call relationships and signatures in the slice. Furthermore, `get_function_source` paginates at 8,000 characters (roughly 100–200 lines) per call; for large functions such as `sqlite3Select` (>1,000 lines), reading the complete body consumes multiple turns, potentially biasing the agent toward superficial inspection of monolithic functions. A (+src) text variant injecting full function source was not evaluated. *Unevaluated graph features.* None of the six benchmark questions specifically concerns a taint vulnerability or struct field hierarchy bug. Consequently, the `get_type_hierarchy` and `get_taint_paths` tools, along with the 84 compile-time `taint_reaches` edges, are not targeted by any of the six questions. *Nemotron ablation exclusion.* When evaluating Nemotron-3-Ultra-550B on tool-ablation subsets, the NVIDIA API repeatedly returned empty responses, short non-JSON strings, or malformed outputs across queries. Because stable ablation runs could not be completed under this API setup, ablation results are reported for Qwen3-Coder-30B only. *Hosting differences and output truncation.* Qwen3-Coder-30B was served locally via LM Studio (the local model's quantisation level was not recorded), whereas Nemotron-3-Ultra-550B was accessed via hosted inference; quantization artefacts may affect reasoning precision. Under Nemotron-3-Ultra-550B SYN-TEXT on Q1, the response exceeded the 3,000 completion-token limit and truncated mid-stream, producing an invalid JSON parse; this run is counted as a miss, which lowers Nemotron-3-Ultra-550B SYN-TEXT's top-3 count in Table 3. *Condition comparability.* The TEXT conditions are not token-matched: SYN slices are smaller than CT slices (2,849 vs. 4,405 tokens for `sqlite3Select`). In addition, SYN-TOOL had 5 tools and CT-TOOL 7; 310 of the SYN graph's 772 call edges were inferred by heuristic. *Sample size and variance.* With six questions and one run per baseline condition at temperature 0, no variance estimate is available, and a one-question difference is not evidence of a systematic effect. *Ground truth determination and fix-build verification.* As noted in Section 3, for Q3 and Q4 the bug was reproduced on the vulnerable build, but the fix-build differential was not run; ground truth follows decision-log rule D14 and was agreed by both authors (for Q4 the upstream fix also modifies `sqlite3ErrorMsg`, outside the analysed files).

External validity. Our study covers one codebase (SQLite 3.30.1), six benchmark questions, and two models. Five of the six benchmark questions share one entry function (`sqlite3Select`) and half involve callbacks. The 31 % callback-gap share is therefore not representative of SQLite as a whole. Results may not generalise to other programming languages, codebase scales, bug classes, or LLM architectures. SQLite's function-pointer-heavy architecture means the callback-gap failure rate may be higher than in object-oriented or functional codebases.

Construct validity. The not-in-graph rate is an upper bound on model fabrication because function-pointer structural gaps also contribute absent edges. We use *not supported by the graph* rather than *hallucinated* to reflect this ambiguity. The 15-turn cap and forced-final mechanism may suppress answers the model would eventually reach with more turns.

8 Related Work

Spectrum-based fault localisation. SBFL methods rank statements or functions by suspiciousness scores derived from pass/fail test-coverage data. Tarantula [Jones and Harrold 2005] introduced the coefficient refined by many later techniques. A broad survey is given by Wong et al. [2016]. Our work differs fundamentally: we use neither test suites nor execution coverage; the LLM reasons directly from a natural-language symptom description and optional call-graph context.

LLMs for code evaluation and program repair. LLMs have been extensively evaluated on code generation and automated program repair [Liu et al. 2023; Sobania et al. 2023]. While benchmark evaluations such as EvalPlus [Liu et al. 2023] assess standalone generation correctness, automated repair tools prompt models iteratively with compiler feedback or test outcomes. Our study focuses on the upstream task of identifying the faulty function prior to repair, investigating whether call-graph representations provide sufficient topological guidance to isolate root causes in complex C software.

Repository navigation agents and context engineering. Recent work investigates how large language models can navigate repository-scale codebases using structured tools. Most closely related to our agent architecture, Yildiz et al. [2025] introduce the JitVul benchmark, deploying ReAct-style agents equipped with interprocedural context to detect vulnerabilities in C repositories. In an empirical investigation of context engineering, Li et al. [2026] demonstrate that "one size does not fit all" when retrieving repository context: while navigation-based agents dynamically explore dependencies, navigation-based approaches incur 10–20× higher cost. Our empirical findings directly mirror this cost trade-off, showing a 10–23× prompt-token multiplier for tool-navigating agents over static BFS text slices, while providing an edge-level audit of the structural gaps and cited edge validity inherent to static call graphs on large C systems.

LLM agents with structured tool use. The ReAct framework [Yao et al. 2023] established the reason-act-observe pattern for autonomous tool-using agents. We apply this pattern to static call-graph exploration, ablating the available tool primitives (call chains, type hierarchies, taint paths, raw source retrieval) to isolate which specific tools assist localisation and whether tool access reduces the citing of edges not supported by the underlying call graph.

9 Conclusion

We presented a controlled empirical study of call-graph-guided LLM fault localisation on the SQLite 3.30.1 codebase across nine conditions and two models. In the 15-turn baseline, graph context provided only marginal exact-accuracy gains: neither model exceeded one correct answer out of five (excluding Q2) or six (Nemotron). Of the 43 wrong answers across all eight graph conditions, 15 (35 %) stem from function-pointer callback gaps that make targets unreachable along static call edges, and 25 (58 %) name a different graph node. Neither graph representation (compile-time Clang vs. syntactic tree-sitter) demonstrated superior accuracy. Text slices match tool conditions in exact accuracy (and remain within one question in top-3 coverage) at 10–12× lower token cost on Qwen3-Coder-30B and 16–23× lower on Nemotron-3-Ultra-550B. Furthermore, caller/callee tools were associated with a 2.4× lower rate of edges not supported by the graph compared to source-only navigation (12 % vs. 28 %), suggesting that edge-traversal tools ground the agent's cited reasoning paths even when they do not alter the final answer. In sensitivity probes (designed after the baseline results were known), extending the turn horizon to 30 enabled agents equipped with source reading to locate callback-separated targets on both graphs, though at substantial token expense.

Future work should address three promising directions: (i) incorporating function-pointer recovery and dynamic dispatch recording to raise the structural reachability ceiling, (ii) supplementing graph tools with lexical code-search (e.g., regex/grep) to directly locate error messages and identifiers appearing in symptoms, and (iii) designing adaptive source-paging mechanisms capable of summarizing monolithic functions like `sqlite3Select` (>1,000 lines) without consuming excessive agent turns.

10 Data Availability

The source code, build scripts for both call-graph databases, evaluation harness, pre-computed result JSONL files, decision logs, and analysis scripts are available in the supplementary repository. The repository includes reproduction instructions, exact dependency specifications, and full benchmark question definitions. No upstream bug reports or patches were filed against SQLite as part of this work.

References

James A. Jones and Mary Jean Harrold. 2005. Empirical Evaluation of the Tarantula Automatic Fault-Localization Technique. In *Proc. 20th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 273–282. doi:10.1145/1101908.1101949

Yichen Li, Qiye Lin, Yun Peng, Zhihan Jiang, Jinyang Liu, Chaozheng Wang, Yintong Huo, and Cuiyun Gao. 2026. One Size Does Not Fit All: Revisiting Code Context Engineering for Repository-Level Code Generation. *Proc. ACM Softw. Eng. (FSE)* (2026). doi:10.1145/3808138

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In *Proc. 37th Conference on Neural Information Processing Systems (NeurIPS)*.

Dominik Sobania, Martin Briesch, Carol Hanna, and Justyna Petke. 2023. An Analysis of the Automatic Bug Fixing Performance of ChatGPT. In *Proc. IEEE/ACM International Workshop on Automated Program Repair (APR)*. 23–30.

SQLite Development Team. 2024. SQLite: A Self-Contained, Serverless SQL Database Engine. https://www.sqlite.org.

W. Eric Wong, Ruizhi Gao, Yihao Li, Rui Abreu, and Franz Wotawa. 2016. A Survey on Software Fault Localization. *IEEE Transactions on Software Engineering* 42, 8 (2016), 707–740. doi:10.1109/TSE.2016.2521368

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proc. 11th International Conference on Learning Representations (ICLR)*.

Alperen Yildiz, Sin G. Teo, Yiling Lou, Yebo Feng, Chong Wang, and Dinil Mon Divakaran. 2025. Benchmarking LLMs and LLM-Based Agents in Practical Vulnerability Detection for Code Repositories. In *Proc. 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*. 30848–30865. doi:10.18653/v1/2025.acl-long.1490